

**UNIVERSITATEA POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII ȘI
TEHNOLOGIA INFORMAȚIEI**

Sistem de transmitere si recepție a codului Morse

**Student: Gae Darius – Andrei
Grupa: 422E**

2025-2026

CUPRINS

Introducere	4
Capitolul 1: Descrierea generală a sistemului	5
1.1 Prezentarea sistemului de comunicații Morse.....	5
1.2 Modul de funcționare.....	6
Capitolul 2: Proiectare și implementare	8
2.1 Etapa de proiectare.....	8
2.2 Semnificația porturilor.....	9
2.3 Implementarea codului sursă	11
2.3.1 Declararea variabilelor utilizate in program.....	11
2.3.2 Implementarea cazurilor.....	12
Capitolul 3: Testarea funcționalității sistemului	14
3.1 Testarea transmisiei și conversiei Morse	14
3.2 Testarea recepției și a cazului special SOS	19
Concluzii	21
Bibliografie	22
Anexe	23

LISTA FIGURILOR

Fig. 1.1 Schema bloc a sistemului Morse	5
Fig. 1.2 Diagrama cazurilor de utilizare a sistemului	7
Fig. 2.1 Diagrama secvențială corespunzătoare codului sursă	12
Fig. 3.1 Valori PORT-uri la START	14
Fig. 3.2 Stare IDLE – LED IDLE activ (PORTB.2)	15
Fig. 3.3 Stare COMPUNERE – LED COMPUNERE activ (PORTB.3)	15
Fig. 3.4 Salvare caracter ASCII în buffer (PINA.1)	16
Fig. 3.5 Ștergere caracter din buffer (PINA.2)	17
Fig. 3.6 Stare TRANSMISIE – LED TRANSMISIE activ (PORTB.4)	17
Fig. 3.7 Stare TRANSMISIE – Afișare semnale LED Punct/Linie	18
Fig. 3.8 Revenirea automată în IDLE la finalul transmisiei	18
Fig. 3.9 Revenirea manuală în IDLE la apăsarea butonului MOD	19
Fig 3.10 receptia unei litere.....	20
Fig 3.11 mesaj special SOS.....	20

LISTA TABELELOR

Tabelul 2.1 Configurarea portului A (Intrări Control)	8
Tabelul 2.2 Configurarea portului B (Ieșiri Semnalizare)	8
Tabelul 2.3 Configurarea portului C (Intrare Recepție Morse)	8
Tabelul 2.4 Configurarea portului D (Intrare Date ASCII)	9
Tabelul 2.5 Semnificația biților din Portul A	9
Tabelul 2.6 Semnificația biților din Portul B	10
Tabelul 2.7 Semnificația biților din Portul C	10
Tabelul 2.8 Semnificația biților din Portul D	11

INTRODUCERE

Prezentul proiect are ca scop proiectarea și implementarea unui sistem de comunicații bidirecțional bazat pe alfabetul Morse, utilizând platforma de dezvoltare bazată pe microcontrolerul **ATmega164P**. Lucrarea explorează interfațarea hardware-software pentru a crea un terminal de transmisie-recepție eficient.

Obiectivele generale ale proiectului vizează:

- Realizarea unui modul de transmisie capabil să convertească caracterele ASCII în semnale luminoase (puncte și linii) conform standardului internațional Morse.
- Implementarea unui sistem de recepție asincron, care să permită captarea și stocarea automată a mesajelor externe fără a întrerupe activitatea curentă a utilizatorului.

Metodologia folosită a presupus o abordare structurată, pornind de la definirea unei **Mașini de Stare Finite (FSM)** pentru controlul logicii aplicației. Pentru partea de dezvoltare software, s-a utilizat limbajul C, punându-se un accent deosebit pe **programarea bazată pe evenimente**. Aceasta a implicat configurarea registrelor pentru întreruperi externe și gestionarea porturilor de intrare/ieșire (I/O) într-un mod care să permită procesarea în paralel a sarcinilor.

Contribuția personală a constat în proiectarea unei arhitecturi **Full-Duplex logic**, eliminând limitările sistemelor secvențiale clasice. O realizare semnificativă o reprezintă dezvoltarea unei rutine de serviciu al întreruperilor (ISR) care monitorizează permanent portul de recepție, asigurând astfel o autonomie completă a procesului de ascultare. De asemenea, am implementat un sistem de gestionare a memoriei prin utilizarea a două buffer-e dedicate, prevenind astfel suprapunerea datelor proprii cu cele primite.

Rezultatele obținute confirmă funcționalitatea sistemului ca terminal de comunicație fiabil. Dispozitivul permite compunerea mesajelor, corectarea acestora în timp real și transmiterea lor, reușind simultan să decodeze și să salveze mesaje externe. Produsul final reprezintă o soluție tehnică integrată ce demonstrează aplicabilitatea conceptelor de procesare asincronă în sistemele embedded moderne.

CAPITOLUL 1: DESCRIEREA GENERALĂ A SISTEMULUI

1.1 PREZENTAREA SISTEMULUI

Sistemul descris are rolul de a asigura comunicarea bidirecțională între doi utilizatori utilizând **codul Morse**. Identificarea și prelucrarea datelor se bazează pe durata semnalelor electrice (puncte și linii), care sunt convertite în caractere ASCII. Pentru această funcționalitate, sunt utilizate intrări digitale sub formă de **butoane** și **switch-uri**, iar feedback-ul este oferit prin **LED-uri** de stare și de transmisie.

Informațiile introduse de utilizator sunt prelucrate de către microcontrolerul **ATmega164P**, care acționează ca un transceiver. Mesajul compus este stocat într-un buffer local înainte de a fi transmis sub formă de impulsuri luminoase și electrice. În paralel, sistemul monitorizează portul de recepție pentru a identifica semnale externe, asigurând un regim de funcționare de tip **Full-Duplex logic**.

Sistemul este alcătuit din următoarele componente:

- **Microcontroler ATmega164P** – Unitatea centrală care gestionează prioritizarea întreruperilor, scanarea butoanelor, temporizarea semnalelor Morse și managementul memoriei (buffer-e).
- **Switch-uri (SW0-SW6)** – Utilizate pentru setarea manuală a codului ASCII corespunzător literei dorite.
- **Butoane de control (MOD, SAVE, TRANSMIT, DELETE)** – Interfața utilizatorului pentru navigarea între stările de așteptare, compunere și transmisie.
- **LED-uri de stare (PB2, PB3, PB4)** – Oferă feedback vizual asupra modului de lucru curent (IDLE, COMPUNERE sau TRANSMISIE).
- **LED-uri de ieșire (PB0, PB1)** – Transformă datele digitale în semnale vizuale Morse (Punct și Linie) pentru transmisia către partener.
- **Intrări de recepție (PC0, PC1)** – Canale dedicate pentru ascultarea asincronă a semnalelor venite de la un dispozitiv extern.

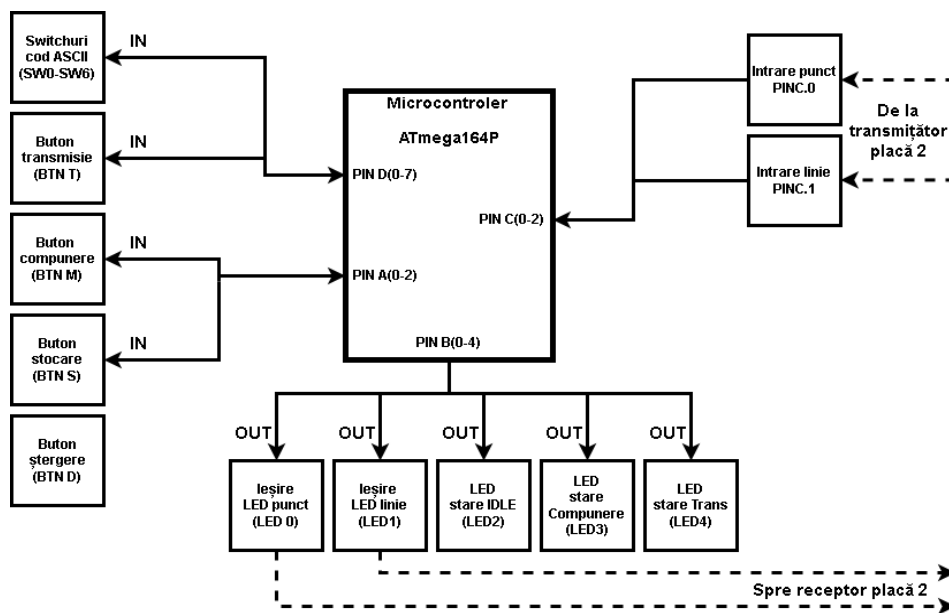


Fig. 1.1 Schema bloc a sistemului

1.2 MOD DE FUNCȚIONARE

Sistemul este coordonat de un **automat de stare finit** (FSM) care gestionează tranzițiile între modurile de operare, fiind dublat de un mecanism de întreruperi pentru recepția datelor. Logica de funcționare urmează următoarea succesiune:

1. Pornirea și Inițializarea (START): La alimentarea plăcii, sistemul execută configurarea porturilor și activează întreruperile globale. Din acest moment, procesorul rulează simultan două fire logice: cel de control (interfața) și cel de ascultare (recepția asincronă).

2. Starea de Așteptare (IDLE): Sistemul începe în această stare, semnalizată prin **LED-ul PB2**. Aici, microcontrolerul scanează permanent butoanele de control. Aceasta este starea de repaus în care se revine după fiecare acțiune finalizată.

3. Accesarea Modulului de Compunere (MOD): Prin apăsarea butonului **MOD**, sistemul trece în faza de editare (**LED PB3 aprins**). Utilizatorul poate seta codul ASCII al caracterelor folosind switch-urile de pe portul D.

4. Salvarea în Buffer (SAVE): După setarea unei litere, apăsarea butonului **SAVE** adaugă caracterul respectiv în memoria temporară (Buffer 1). Sistemul rămâne în modul de compunere pentru a permite scrierea unui mesaj întreg.

5. Corecția Erorilor (DELETE): Dacă utilizatorul observă o greșeală după salvare, poate apăsa butonul **DELETE**. Acesta elimină ultimul caracter introdus din Buffer 1, menținând sistemul în starea de compunere pentru a permite rescrierea imediată a literei corecte.

6. Transmisia Mesajului (TRANSMIT): La apăsarea butonului **TRANSMIT**, sistemul intră în starea de emisie (**LED PB4 aprins**). Caracterele din Buffer 1 sunt convertite în cod Morse și trimise secvențial prin pinii **PB0 (Punct)** și **PB1 (Linie)**.

7. Revenirea Automată (AUTO-RETURN): Imediat ce ultimul bit al mesajului a fost transmis, sistemul execută o rutină de curățare a memoriei și revine automat în starea **2 (IDLE)**. Nu este necesară nicio comandă manuală de resetare.

8. Anularea Sesiunii (CANCEL): În orice moment al compunerii (starea 3), dacă se apasă din nou butonul **MOD**, sistemul abandonează mesajul curent și forțează revenirea în starea **2 (IDLE)**.

9. Recepția în Fundal (ISR): Indiferent de stările de mai sus (1-8), orice semnal extern sosit pe pinii **PINC0/PINC1** este captat instantaneu. Datele sunt procesate și stocate într-un **Buffer 2** separat, fără a interfera cu acțiunile utilizatorului pe butoane.

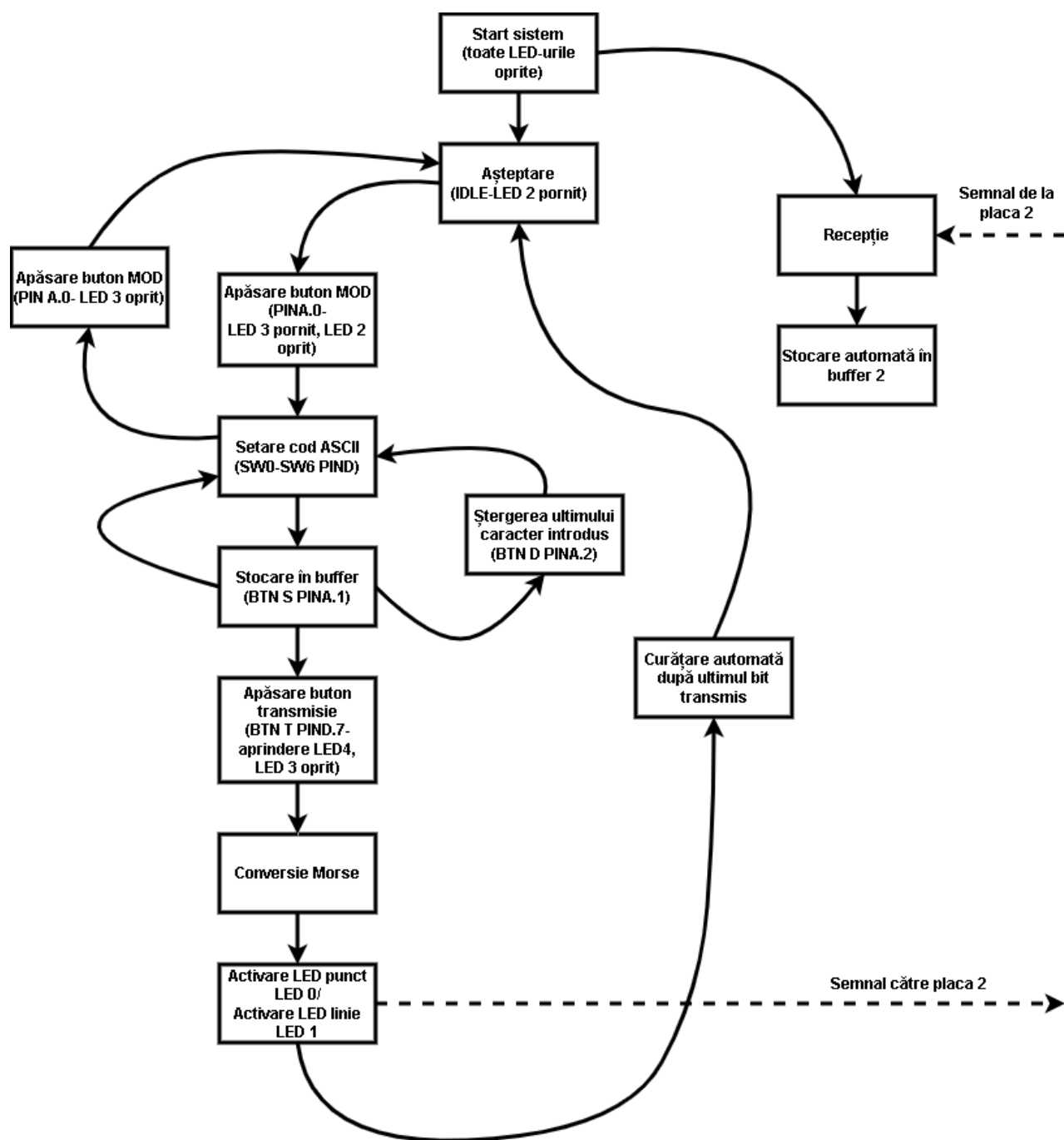


Fig. 1.2 Diagrama cazurilor de utilizare a sistemului

CAPITOLUL 2: PROIECTARE ȘI IMPLEMENTARE

2.1 ETAPA DE PROIECTARE

Sistemul este proiectat pentru a converti caractere ASCII (introduse prin switch-uri) în semnale luminoase și sonore conform alfabetului Morse, permițând totodată recepția și stocarea mesajelor primite de la un alt sistem.

Pentru scrierea codului, am utilizat aplicația **CodeVision AVR**, codul fiind scris în limbajul de programare C. La crearea proiectului, au fost realizate următoarele setări:

- **Chip** → **Chip Settings** → **Chip**: am selectat opțiunea ATmega164P. Din aceeași secțiune, am modificat frecvența de ceas la **10 MHz**, aceasta influențând direct viteza de execuție a instrucțiunilor.
- **Port** → **Port Settings**: fiecare port a fost configurat în funcție de rolul său în procesul de programare și control al componentelor hardware.

PORT A								
Data Direction	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
	In	In	In	In	In	In	In	In
Pullup/Output Value	P	P	P	T	T	T	T	T

Tabelul 2.1 Configurarea portului A

PORT B								
Data Direction	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
	OUT	OUT	OUT	OUT	OUT	In	In	In
Pullup/Output Value	0	0	0	0	0	T	T	T

Tabelul 2.2 Configurarea portului B

PORT C								
Data Direction	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
	In	In	In	In	In	In	In	In
Pullup/Output Value	P	P	T	T	T	T	T	T

Tabelul 2.3 Configurarea portului C

PORT D								
Data Direction	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
	In	In	In	In	In	In	In	In
Pullup/Output Value	P	P	P	P	P	P	P	P

Tabelul 2.4 Configurarea portului D

- **Timers/Counters** → **Clock Value** → **Timer0** → **9.766 kHz**: este utilizat un semnal de ceas de 9.766 kHz pentru Timer0. Bifarea opțiunii **Overflow Interrupt** permite generarea unei întreruperi de fiecare dată când Timer0 ajunge la valoarea maximă și revine la zero. Aceasta întrerupere este tratată printr-o funcție specială numită Interrupt Service Routine (**ISR**). În plus, valoarea **Timer Value** → **3Ch (în hexazecimal)** definește punctul de la care începe număratoarea, astfel încât întreruperea să fie declanșată aproximativ la fiecare **20 ms**.

2.2 SEMNIFICAȚIA PORTURILOR

Fiecare port al microcontrolerului ATmega164P are un rol bine definit în gestionarea transmiterii și recepției codului Morse, asigurând funcționarea simultană a celor două module.

Portul A – Controlul Stării de Transmisie

Portul A este utilizat pentru interfațarea cu butoanele de control ale utilizatorului.

- **PINA.0 (MOD)**: Comută între starea de repaus (IDLE) și starea de compunere a mesajului.
- **PINA.1 (SAVE)**: Salvează caracterul ASCII setat pe PIND în bufferul de transmisie.
- **PINA.2 (DELETE)**: Permite ștergerea ultimului caracter introdus în buffer, dacă sistemul este în modul compunere.

PORT A							
Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
Buton MOD	Buton SAVE	Buton DELETE					

Tabelul 2.5 Semnificatia bitilor din portul A

Portul B – Semnalizare și Alerte (Ieșiri)

Portul B controlează indicatorii vizuali. Biții 0 și 1 au rol dublu: semnalizare Morse și alertă SOS.

- **PORTB.0 (LED Punct):** Pulsează la transmisia unui punct. Clipește la recepția unui SOS.
- **PORTB.1 (LED Linie):** Pulsează la transmisia unei linii. Clipește sincron cu Bit 0 la SOS.
- **PORTB.2 (LED IDLE):** Aprins când sistemul este în așteptare (Starea 0).
- **PORTB.3 (LED COMPUNERE):** Aprins în timpul editării mesajului (Starea 1).
- **PORTB.4 (LED TRANSMISIE):** Aprins pe toată durata trimiterii mesajului (Starea 2).

PORT B							
Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
LED punct	LED linie	LED IDLE	LED COMP	LED TRANS			

Tabelul 2.6 Semnificatia bitilor din portul B

Portul C – Recepție Semnal (Intrări)

Portul C este utilizat pentru eșantionarea semnalului Morse extern prin rutina de întrerupere (ISR).

- **PINC.0 și PINC.1:** Pini de intrare pentru semnalul Morse recepționat. Sistemul monitorizează acești pini la fiecare **20.07 ms** pentru a identifica durata stării active (0 logic).

PORT C							
Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
Intrare Morse 1	Intrare Morse 2						

Tabelul 2.7 Semnificatia bitilor din portul C

Portul D – Intrare Date ASCII (Intrări)

Portul D preia codul ASCII al caracterului ce urmează a fi salvat și gestionează declanșarea transmisiei.

- **PIND (Biții 0-6):** Reprezintă valoarea în format binar a caracterului ASCII (ex: 0x41 pentru 'A').

- **PIND.7 (TRANSMIT):** Declanșează trecerea din starea de compunere în starea de transmisie.

PORT D							
Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
Valoare ASCII	Valoare ASCII	Valoare ASCII	Valoare ASCII	Valoare ASCII	Valoare ASCII	Valoare ASCII	Buton TRANSMIT

Tabelul 2.8 Semnificatia bitilor din portul D

2.3 IMPLEMENTAREA CODULUI SURSĂ

2.3.1 DECLARAREA VARIABILELOR UTILIZATE ÎN PROGRAM

Pentru implementarea funcționalităților sistemului de comunicare Morse, au fost definite următoarele categorii de variabile:

Controlul Automatului Finit (FSM):

- **unsigned char Q = 0;** – Reprezintă starea curentă a sistemului:
 1. **0 – Așteptare (IDLE):** Sistemul este pregătit pentru o nouă comandă.
 2. **1 – Compunere:** Utilizatorul introduce caractere ASCII în buffer.
 3. **2 – Transmisie:** Mesajul este convertit și transmis vizual prin LED-uri.

Gestionarea Bufferului de Transmisie:

- **unsigned char buffer_tx[20];** – Vector care stochează caracterele ASCII introduse de utilizator (capacitate 20 caractere).
- **int index_tx = 0;** – Contor care indică poziția curentă în bufferul de transmisie.

Gestionarea Recepției și Decodării (Background):

- **unsigned char buffer_rx[20];** – Buffer circular pentru stocarea mesajelor recepționate extern.
- **unsigned char temp_morse[6];** – Stochează secvența de puncte și linii pentru o singură literă în curs de decodare.
- **int temp_index = 0;** – Indice pentru asamblarea simbolurilor Morse recepționate.

Variabile de Temporizare și Alerte:

- **unsigned char contor_20ms = 0;** – Contorizează durata semnalului activ (eșantioane de ~20ms) pentru a distinge între punct și linie.
- **unsigned char pauza_contor = 0;** – Măsoară durata lipsei semnalului pentru a identifica pauzele între litere sau cuvinte.
- **unsigned char SOS_activ = 0;** – Indicator logic activat la recepția secvenței SOS; blochează transmisia și activează alerta vizuală.

2.3.2 IMPLEMENTAREA CAZURILOR (DIAGRAMA FSM)

Sistemul este structurat pe baza unui **automat finit cu 3 stări**, implementat printr-o instrucțiune switch-case. Trecerea de la o stare la alta este controlată de intrările de pe Portul A și Portul D.

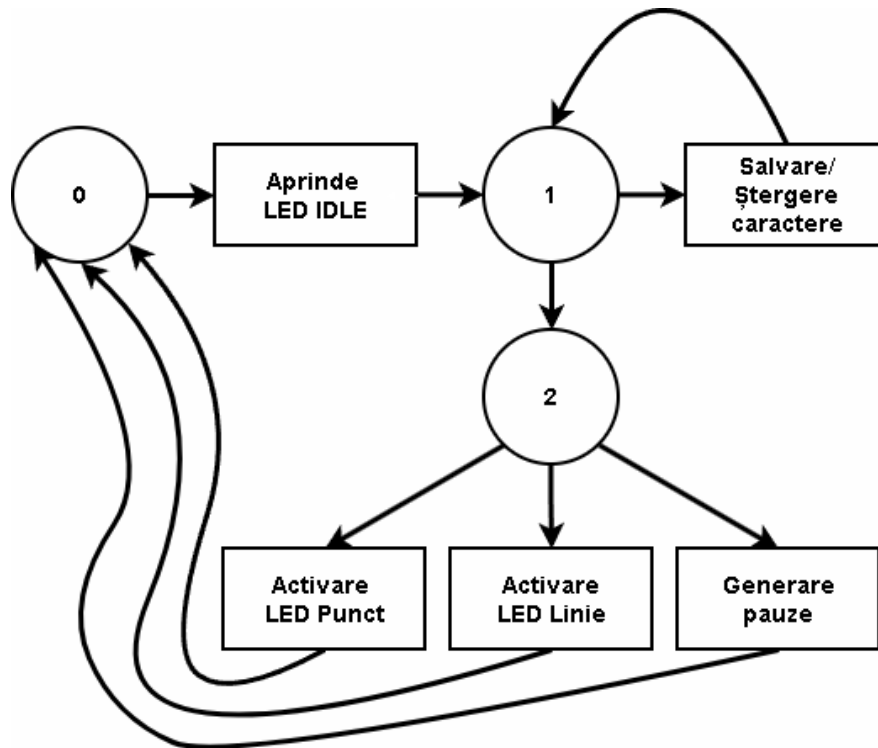


Fig. 2.1 Diagrama secvențială corespunzătoare codului sursă

Starea 0 – Așteptare (IDLE) În această stare, sistemul consumă resurse minime și așteaptă interacțiunea utilizatorului. LED-ul de IDLE (PORTB.2) este aprins.

```

case 0: // IDLE
    PORTB.2 = 1; // LED IDLE activ
    if (PINA.0 == 0) { // Apasare buton MOD
        delay_ms(200);
        PORTB.2 = 0;
        Q = 1; // Trecere in starea Compunere
    }
    break;

```

Starea 1 – Compunere Utilizatorul utilizează switch-urile de pe Portul D pentru a seta codul ASCII al unei litere și butonul SAVE (PINA.1) pentru a o adăuga în buffer.

```
case 1: // COMPUNERE
    PORTB.3 = 1; // LED Compunere activ
    if (PINA.1 == 0) { // Buton SAVE
        if (index_tx < 20) {
            buffer_tx[index_tx] = PIND & 0x7F; // Citeste ASCII
            index_tx++;
        }
        delay_ms(250);
    }
    if (PIND.7 == 0 && SOS_activ == 0) { // Buton TRANSMIT
        Q = 2; // Trecere in starea Transmisie
    }
    break;
```

Starea 2 – Transmisie Sistemul parcurge buffer-ul și apelează funcțiile de conversie. Fiecare caracter este transformat în semnale electrice pe biții PORTB.0 (Punct) și PORTB.1 (Linie).

```
case 2: // TRANSMISIE
    PORTB.4 = 1; // LED Transmisie activ
    executa_transmisie(); // Functia care goleste buffer-ul si "clipecte" Morse
    Q = 0; // Revenire automata in IDLE
    break;
```

Codul sursă complet al programului este inclus în Anexa 2.

CAPITOLUL 3. TESTAREA FUNCȚIONALITĂȚII SISTEMULUI

Verificarea sistemului de comunicație Morse s-a realizat prin simulare în mediul **AVR Studio 4**, urmărind evoluția variabilelor în fereastra *Watch* și starea pinilor în *I/O View*.

Starea 1 – Pornire și Inițializare (START)

Aceasta este faza de configurare hardware executată o singură dată la alimentarea plăcii.

- **Acțiuni:** Configurarea registrelor de direcție: **DDRA** (Intrări butoane), **DDRB** (Ieșiri LED/Morse), **DDRC** (Intrări senzori), **DDRD** (Intrări date/switch-uri).
- **Logică:** Se activează rezistențele de pull-up pe intrări (**PORTA**, **PORTC**, **PORTD**) și se inițializează **Timer0** pentru a genera o bază de timp de **20.07 ms** prin încărcarea valorii **0x3C** în **TCNT0**.
- **Tranziție:** Programul intră în bucla infinită **while(1)**, pornind implicit cu **Q = 0**.

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINA	0x00 (0x20)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTA	0x02 (0x22)	0x07	☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒
DDRB	0x04 (0x24)	0x1F	☐ ☐ ☐ ☒ ☒ ☒ ☒ ☒
PINB	0x03 (0x23)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTB	0x05 (0x25)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
DDRC	0x07 (0x27)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINC	0x06 (0x26)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTC	0x08 (0x28)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRD	0x0A (0x2A)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PIND	0x09 (0x29)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTD	0x0B (0x2B)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Name	Value	Type	Location
Q	0 ''	unsigned char	R4
index_tx	0	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
contor_20ms	0 ''	unsigned char	R3

Fig. 3.1 Valori PORT-uri la START

2. Starea de Așteptare (IDLE) — case 0

Sistemul se află în repaus, pregătit pentru utilizare.

- **Semnalizare:** Se activează LED-ul de stare IDLE prin instrucțiunea **PORTB.2 = 1**.
- **Condiție de ieșire:** Microcontrolerul scanează pinul **PINA.0**. Dacă se detectează valoarea **0** (apăsare buton MOD), se execută un **delay_ms(200)** pentru debouncing.
- **Tranziție:** **PORTB.2** devine **0** și variabila de stare devine **Q = 1**.

Name	Address	Value	Bits	
DDRA	0x01 (0x21)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PINA	0x00 (0x20)	0x07	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
PORTA	0x02 (0x22)	0x07	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
DDRB	0x04 (0x24)	0x1F	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒	
PINB	0x03 (0x23)	0x04	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
PORTB	0x05 (0x25)	0x04	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
DDRC	0x07 (0x27)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PINC	0x06 (0x26)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PORTC	0x08 (0x28)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
DDRD	0x0A (0x2A)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PIND	0x09 (0x29)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PORTD	0x0B (0x2B)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒	

Name	Value	Type	Location
Q	0 ''	unsigned char	R4
index_tx	0	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
contor_20ms	39 '''	unsigned char	R3

Fig. 3.2 stare IDLE - LED IDLE activ

3. Accesarea Modului de Compunere (MOD) — case 1

Sistemul trece în faza de pregătire a mesajului ce urmează a fi transmis.

- **Semnalizare:** Se activează indicatorul vizual pentru editare: **PORTB.3 = 1**.
- **Funcționare:** În acest mod, sistemul „ascultă” comenzile de salvare, ștergere sau transmitere, permițând utilizatorului să interacționeze cu butoanele de pe **PORTA** și switch-urile de pe **PORTD**.

Name	Address	Value	Bits	
DDRA	0x01 (0x21)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PINA	0x00 (0x20)	0x06	☐ ☐ ☐ ☐ ☐ ☒ ☒	
PORTA	0x02 (0x22)	0x07	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
DDRB	0x04 (0x24)	0x1F	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒	
PINB	0x03 (0x23)	0x08	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
PORTB	0x05 (0x25)	0x08	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
DDRC	0x07 (0x27)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PINC	0x06 (0x26)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PORTC	0x08 (0x28)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒	
DDRD	0x0A (0x2A)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	
PIND	0x09 (0x29)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒	
PORTD	0x0B (0x2B)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒	

Name	Value	Type	Location
Q	1 'I'	unsigned char	R4
index_tx	0	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
contor_20ms	119 'w'	unsigned char	R3

Fig. 3.3 stare COMPUNERE - LED COMPUNERE activ

4. Salvarea în Buffer (SAVE)

Preluarea caracterelor setate manual în memoria sistemului.

- **Condiție:** Detecția apăsării butonului SAVE pe **PINA.1 == 0**.

- **Logică:** Dacă buffer-ul nu este plin (**index_tx < MAX_BUFFER**), se citește valoarea de pe **PIND** (mascată cu **0x7F** pentru a păstra formatul ASCII de 7 biți).
- **Efect:** Valoarea este stocată în **buffer_tx[index_tx]** și **indexul** este incrementat.

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINA	0x00 (0x20)	0x05	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
PORTA	0x02 (0x22)	0x07	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRB	0x04 (0x24)	0x1F	☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒
PINB	0x03 (0x23)	0x08	☐ ☐ ☐ ☐ ☒ ☐ ☐ ☐
PORTB	0x05 (0x25)	0x08	☐ ☐ ☐ ☐ ☒ ☐ ☐ ☐
DDRC	0x07 (0x27)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINC	0x06 (0x26)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTC	0x08 (0x28)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRD	0x0A (0x2A)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PIND	0x09 (0x29)	0xC7	☒ ☒ ☒ ☐ ☐ ☐ ☐ ☐
PORTD	0x0B (0x2B)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Name	Value	Type	Location
Q	1 '1'	unsigned char	R4
index_tx	1	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
[0]	71 'G'	unsigned char	0x0200 [SRAM]
[1]	0 ''	unsigned char	0x0201 [SRAM]
[2]	0 ''	unsigned char	0x0202 [SRAM]
[3]	0 ''	unsigned char	0x0203 [SRAM]
[4]	0 ''	unsigned char	0x0204 [SRAM]
[5]	0 ''	unsigned char	0x0205 [SRAM]
[6]	0 ''	unsigned char	0x0206 [SRAM]
[7]	0 ''	unsigned char	0x0207 [SRAM]
[8]	0 ''	unsigned char	0x0208 [SRAM]
[9]	0 ''	unsigned char	0x0209 [SRAM]
[10]	0 ''	unsigned char	0x020A [SRAM]
[11]	0 ''	unsigned char	0x020B [SRAM]
[12]	0 ''	unsigned char	0x020C [SRAM]
[13]	0 ''	unsigned char	0x020D [SRAM]
[14]	0 ''	unsigned char	0x020E [SRAM]
[15]	0 ''	unsigned char	0x020F [SRAM]
[16]	0 ''	unsigned char	0x0210 [SRAM]
[17]	0 ''	unsigned char	0x0211 [SRAM]
[18]	0 ''	unsigned char	0x0212 [SRAM]
[19]	0 ''	unsigned char	0x0213 [SRAM]
contor_20ms	144 ''	unsigned char	R3

Fig. 3.4 salvare in buffer

5. Corecția Erorilor (DELETE)

Permite utilizatorului să modifice mesajul înainte de a fi trimis.

- **Condiție:** Detecția apăsării butonului DELETE pe **PINA.2 == 0**.
- **Logică:** Dacă există caractere salvate (**index_tx > 0**), se decrementează indexul de scriere, eliminând ultimul caracter din coada de transmisie.

Name	Address	Value	Bits	
DDRA	0x01 (0x21)	0x00	□□□□□□□□	
PINA	0x00 (0x20)	0x01	□□□□□□□■	
PORTA	0x02 (0x22)	0x07	□□□□□□■□	
DDRB	0x04 (0x24)	0x1F	□□□□□□■□	
PINB	0x03 (0x23)	0x08	□□□□□□■□	
PORTB	0x05 (0x25)	0x08	□□□□□□■□	
DDRC	0x07 (0x27)	0x00	□□□□□□□□	
PINC	0x06 (0x26)	0x00	□□□□□□□□	
PORTC	0x08 (0x28)	0x03	□□□□□□■□	
DDRD	0x0A (0x2A)	0x00	□□□□□□□□	
PIND	0x09 (0x29)	0xC7	■□□□□□■□	
PORTD	0x0B (0x2B)	0xFF	■□□□□□■□	

Name	Value	Type	Location
Q	1 'g'	unsigned char	R4
index_tx	0	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
[0]	71 'G'	unsigned char	0x0200 [SRAM]
[1]	0 ''	unsigned char	0x0201 [SRAM]
[2]	0 ''	unsigned char	0x0202 [SRAM]
[3]	0 ''	unsigned char	0x0203 [SRAM]
[4]	0 ''	unsigned char	0x0204 [SRAM]
[5]	0 ''	unsigned char	0x0205 [SRAM]
[6]	0 ''	unsigned char	0x0206 [SRAM]
[7]	0 ''	unsigned char	0x0207 [SRAM]
[8]	0 ''	unsigned char	0x0208 [SRAM]
[9]	0 ''	unsigned char	0x0209 [SRAM]
[10]	0 ''	unsigned char	0x020A [SRAM]
[11]	0 ''	unsigned char	0x020B [SRAM]
[12]	0 ''	unsigned char	0x020C [SRAM]
[13]	0 ''	unsigned char	0x020D [SRAM]
[14]	0 ''	unsigned char	0x020E [SRAM]
[15]	0 ''	unsigned char	0x020F [SRAM]
[16]	0 ''	unsigned char	0x0210 [SRAM]
[17]	0 ''	unsigned char	0x0211 [SRAM]
[18]	0 ''	unsigned char	0x0212 [SRAM]
[19]	0 ''	unsigned char	0x0213 [SRAM]
contor_20ms	169 'e'	unsigned char	R3

Fig. 3.5 stergere din buffer

6. Transmisia Mesajului (TRANSMIT)

Declanșarea procesului de emisie a codului Morse.

- Condiție: Detecția butonului TRANSMIT pe **PIND.7 == 0**, cu condiția ca sistemul să nu fie ocupat cu un semnal de urgență (**SOS_activ == 0**).
- Tranziție: Se stinge LED-ul de compunere (**PORTB.3 = 0**) și se trece în starea **Q = 2**.

Name	Address	Value	Bits	
DDRA	0x01 (0x21)	0x00	□□□□□□□□	
PINA	0x00 (0x20)	0x07	□□□□□□■□	
PORTA	0x02 (0x22)	0x07	□□□□□□■□	
DDRB	0x04 (0x24)	0x1F	□□□□□□■□	
PINB	0x03 (0x23)	0x00	□□□□□□□□	
PORTB	0x05 (0x25)	0x00	□□□□□□□□	
DDRC	0x07 (0x27)	0x00	□□□□□□□□	
PINC	0x06 (0x26)	0x00	□□□□□□□□	
PORTC	0x08 (0x28)	0x03	□□□□□□■□	
DDRD	0x0A (0x2A)	0x00	□□□□□□□□	
PIND	0x09 (0x29)	0x47	□□□□□□■□	
PORTD	0x0B (0x2B)	0x7F	□□□□□□■□	

Name	Value	Type	Location
Q	2 ' '	unsigned char	R4
index_tx	1	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
[0]	71 'G'	unsigned char	0x0200 [SRAM]
[1]	0 ''	unsigned char	0x0201 [SRAM]
[2]	0 ''	unsigned char	0x0202 [SRAM]
[3]	0 ''	unsigned char	0x0203 [SRAM]
[4]	0 ''	unsigned char	0x0204 [SRAM]
[5]	0 ''	unsigned char	0x0205 [SRAM]
[6]	0 ''	unsigned char	0x0206 [SRAM]
[7]	0 ''	unsigned char	0x0207 [SRAM]
[8]	0 ''	unsigned char	0x0208 [SRAM]
[9]	0 ''	unsigned char	0x0209 [SRAM]
[10]	0 ''	unsigned char	0x020A [SRAM]
[11]	0 ''	unsigned char	0x020B [SRAM]
[12]	0 ''	unsigned char	0x020C [SRAM]
[13]	0 ''	unsigned char	0x020D [SRAM]
[14]	0 ''	unsigned char	0x020E [SRAM]
[15]	0 ''	unsigned char	0x020F [SRAM]
[16]	0 ''	unsigned char	0x0210 [SRAM]
[17]	0 ''	unsigned char	0x0211 [SRAM]
[18]	0 ''	unsigned char	0x0212 [SRAM]
[19]	0 ''	unsigned char	0x0213 [SRAM]
contor_20ms	191 'z'	unsigned char	R3

Fig. 3.6 stare TRANSMISIE – LED TRANSMISIE activ

7. Procesul de Emisie (Execution) — case 2

Transformarea datelor digitale în semnale luminoase/sonore.

- **Acțiuni:** Se apelează funcția **executa_transmisie()**. Aceasta activează **PORTB.4** (LED TX) și parcurge elementele din **buffer_tx**.
- **Conversie:** Fiecare caracter este trimis prin funcția **trimite_caracter(c)**, care activează secvențial **PORTB.0** (Punct) sau **PORTB.1** (Linie) folosind baza de timp **T_BASE = 100ms**.
- **Tranziție:** După terminarea mesajului, se resetează indexul (**index_tx = 0**) și sistemul revine automat la **Q = 0**.
-

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	□□□□□□□□
PINA	0x00 (0x20)	0x07	□□□□□■□□
PORTA	0x02 (0x22)	0x07	□□□□□■□□
DDRB	0x04 (0x24)	0x1F	□□□□□■□□
PINB	0x03 (0x23)	0x11	□□□□□■□□
PORTB	0x05 (0x25)	0x11	□□□□□■□□
DDRC	0x07 (0x27)	0x00	□□□□□□□□
PINC	0x06 (0x26)	0x00	□□□□□□□□
PORTC	0x08 (0x28)	0x03	□□□□□□□□
DDRD	0x0A (0x2A)	0x00	□□□□□□□□
PIND	0x09 (0x29)	0x41	□■□□□□□□
PORTD	0x0B (0x2B)	0x7F	□■□□□□□□

Name	Value	Type	Location
Q	2 ''	unsigned char	R4
index_tx	1	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
[0]	65 'A'	unsigned char	0x0200 [SRAM]
[1]	0 ''	unsigned char	0x0201 [SRAM]
[2]	0 ''	unsigned char	0x0202 [SRAM]
[3]	0 ''	unsigned char	0x0203 [SRAM]
[4]	0 ''	unsigned char	0x0204 [SRAM]
[5]	0 ''	unsigned char	0x0205 [SRAM]
[6]	0 ''	unsigned char	0x0206 [SRAM]
[7]	0 ''	unsigned char	0x0207 [SRAM]
[8]	0 ''	unsigned char	0x0208 [SRAM]
[9]	0 ''	unsigned char	0x0209 [SRAM]
[10]	0 ''	unsigned char	0x020A [SRAM]
[11]	0 ''	unsigned char	0x020B [SRAM]
[12]	0 ''	unsigned char	0x020C [SRAM]
[13]	0 ''	unsigned char	0x020D [SRAM]
[14]	0 ''	unsigned char	0x020E [SRAM]
[15]	0 ''	unsigned char	0x020F [SRAM]
[16]	0 ''	unsigned char	0x0210 [SRAM]
[17]	0 ''	unsigned char	0x0211 [SRAM]
[18]	0 ''	unsigned char	0x0212 [SRAM]
[19]	0 ''	unsigned char	0x0213 [SRAM]
contor_20ms	71 'G'	unsigned char	R3

Fig. 3.7 stare TRANSMISIE – afisare

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	□□□□□□□□
PINA	0x00 (0x20)	0x07	□□□□□■□□
PORTA	0x02 (0x22)	0x07	□□□□□■□□
DDRB	0x04 (0x24)	0x1F	□□□□□■□□
PINB	0x03 (0x23)	0x00	□□□□□□□□
PORTB	0x05 (0x25)	0x00	□□□□□□□□
DDRC	0x07 (0x27)	0x00	□□□□□□□□
PINC	0x06 (0x26)	0x00	□□□□□□□□
PORTC	0x08 (0x28)	0x03	□□□□□□□□
DDRD	0x0A (0x2A)	0x00	□□□□□□□□
PIND	0x09 (0x29)	0x41	□■□□□□□□
PORTD	0x0B (0x2B)	0x7F	□■□□□□□□

Name	Value	Type	Location
Q	0 ''	unsigned char	R4
index_tx	0	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
[0]	65 'A'	unsigned char	0x0200 [SRAM]
[1]	0 ''	unsigned char	0x0201 [SRAM]
[2]	0 ''	unsigned char	0x0202 [SRAM]
[3]	0 ''	unsigned char	0x0203 [SRAM]
[4]	0 ''	unsigned char	0x0204 [SRAM]
[5]	0 ''	unsigned char	0x0205 [SRAM]
[6]	0 ''	unsigned char	0x0206 [SRAM]
[7]	0 ''	unsigned char	0x0207 [SRAM]
[8]	0 ''	unsigned char	0x0208 [SRAM]
[9]	0 ''	unsigned char	0x0209 [SRAM]
[10]	0 ''	unsigned char	0x020A [SRAM]
[11]	0 ''	unsigned char	0x020B [SRAM]
[12]	0 ''	unsigned char	0x020C [SRAM]
[13]	0 ''	unsigned char	0x020D [SRAM]
[14]	0 ''	unsigned char	0x020E [SRAM]
[15]	0 ''	unsigned char	0x020F [SRAM]
[16]	0 ''	unsigned char	0x0210 [SRAM]
[17]	0 ''	unsigned char	0x0211 [SRAM]
[18]	0 ''	unsigned char	0x0212 [SRAM]
[19]	0 ''	unsigned char	0x0213 [SRAM]
contor_20ms	131 'f'	unsigned char	R3

Fig. 3.8 revenirea in IDLE la finalul transmisiei

8. Anularea Sesiunii (CANCEL)

Revenirea manuală la starea de repaus fără a transmite datele.

- **Condiție:** O a doua apăsare a butonului MOD (**PINA.0 == 0**) în timp ce utilizatorul se află în starea 1.
- **Efect:** Se stinge LED-ul de compunere, se abandonează buffer-ul curent și se forțează **Q = 0**.

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	□□□□□□□□
PINA	0x00 (0x20)	0x06	□□□□□■□□
PORTA	0x02 (0x22)	0x07	□□□□□■□□
DDRB	0x04 (0x24)	0x1F	□□□□□■□■□■
PINB	0x03 (0x23)	0x00	□□□□□□□□
PORTB	0x05 (0x25)	0x00	□□□□□□□□
DDRC	0x07 (0x27)	0x00	□□□□□□□□
PINC	0x06 (0x26)	0x00	□□□□□□□□
PORTC	0x08 (0x28)	0x03	□□□□□□■□
DDRD	0x0A (0x2A)	0x00	□□□□□□□□
PIND	0x09 (0x29)	0xC1	■□□□□□■□
PORTD	0x0B (0x2B)	0x7F	□■□■□■□■□■

Name	Value	Type	Location
Q	0 ''	unsigned char	R4
index_tx	0	int	R6:R5
buffer_tx	[...]	unsigned char[20]	0x0200 [SRAM]
[0]	65 'A'	unsigned char	0x0200 [SRAM]
[1]	0 ''	unsigned char	0x0201 [SRAM]
[2]	0 ''	unsigned char	0x0202 [SRAM]
[3]	0 ''	unsigned char	0x0203 [SRAM]
[4]	0 ''	unsigned char	0x0204 [SRAM]
[5]	0 ''	unsigned char	0x0205 [SRAM]
[6]	0 ''	unsigned char	0x0206 [SRAM]
[7]	0 ''	unsigned char	0x0207 [SRAM]
[8]	0 ''	unsigned char	0x0208 [SRAM]
[9]	0 ''	unsigned char	0x0209 [SRAM]
[10]	0 ''	unsigned char	0x020A [SRAM]
[11]	0 ''	unsigned char	0x020B [SRAM]
[12]	0 ''	unsigned char	0x020C [SRAM]
[13]	0 ''	unsigned char	0x020D [SRAM]
[14]	0 ''	unsigned char	0x020E [SRAM]
[15]	0 ''	unsigned char	0x020F [SRAM]
[16]	0 ''	unsigned char	0x0210 [SRAM]
[17]	0 ''	unsigned char	0x0211 [SRAM]
[18]	0 ''	unsigned char	0x0212 [SRAM]
[19]	0 ''	unsigned char	0x0213 [SRAM]
contor_20ms	151 '-'	unsigned char	R3

Fig. 3.9 revenirea in IDLE la apasarea BTN MOD

9. Recepția în Fundal (ISR) — interrupt [TIM0_OVF]

Monitorizarea liniei de intrare independent de starea automatului finit.

- **Funcționare:** La fiecare **20.07 ms**, timer-ul generează o întrerupere. Se eșantionează pinii **PINC.0** și **PINC.1**.
- **Logică:** Dacă intrarea este **0** logic (activ), se incrementează **contor_20ms**. La dispariția semnalului, în funcție de valoarea acumulată, sistemul poate identifica dacă s-a recepționat un punct (~5 eșantioane) sau o linie (~15 eșantioane).

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINA	0x00 (0x20)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTA	0x02 (0x22)	0x07	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRB	0x04 (0x24)	0x1F	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒
PINB	0x03 (0x23)	0x08	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒
PORTB	0x05 (0x25)	0x08	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒
DDRC	0x07 (0x27)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINC	0x06 (0x26)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
PORTC	0x08 (0x28)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRD	0x0A (0x2A)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PIND	0x09 (0x29)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTD	0x0B (0x2B)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Name	Value	Type	Location
index_ex	2	int	R8:R7
buffer_ex	[...]	unsigned char[20]	0x0214 [SRAM]
[0]	35 '#'	unsigned char	0x0214 [SRAM]
[1]	69 'E'	unsigned char	0x0215 [SRAM]
[2]	0 ''	unsigned char	0x0216 [SRAM]
[3]	0 ''	unsigned char	0x0217 [SRAM]
[4]	0 ''	unsigned char	0x0218 [SRAM]
[5]	0 ''	unsigned char	0x0219 [SRAM]
[6]	0 ''	unsigned char	0x021A [SRAM]
[7]	0 ''	unsigned char	0x021B [SRAM]
[8]	0 ''	unsigned char	0x021C [SRAM]
[9]	0 ''	unsigned char	0x021D [SRAM]
[10]	0 ''	unsigned char	0x021E [SRAM]
[11]	0 ''	unsigned char	0x021F [SRAM]
[12]	0 ''	unsigned char	0x0220 [SRAM]
[13]	0 ''	unsigned char	0x0221 [SRAM]
[14]	0 ''	unsigned char	0x0222 [SRAM]
[15]	0 ''	unsigned char	0x0223 [SRAM]
[16]	0 ''	unsigned char	0x0224 [SRAM]
[17]	0 ''	unsigned char	0x0225 [SRAM]
[18]	0 ''	unsigned char	0x0226 [SRAM]
[19]	0 ''	unsigned char	0x0227 [SRAM]
contor_20ms	0 ''	unsigned char	R13
pausa_contor	15 'I'	unsigned char	R3
temp_morse	[...]	unsigned char[6]	0x0228 [SRAM]
[0]	46 '.,'	unsigned char	0x0228 [SRAM]
[1]	0 ''	unsigned char	0x0229 [SRAM]
[2]	0 ''	unsigned char	0x022A [SRAM]
[3]	0 ''	unsigned char	0x022B [SRAM]
[4]	0 ''	unsigned char	0x022C [SRAM]
[5]	0 ''	unsigned char	0x022D [SRAM]

Fig 3.10 receptia unei litere

Name	Address	Value	Bits
DDRA	0x01 (0x21)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINA	0x00 (0x20)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTA	0x02 (0x22)	0x07	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRB	0x04 (0x24)	0x1F	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒
PINB	0x03 (0x23)	0x08	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒
PORTB	0x05 (0x25)	0x08	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒
DDRC	0x07 (0x27)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PINC	0x06 (0x26)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
PORTC	0x08 (0x28)	0x03	☐ ☐ ☐ ☐ ☐ ☐ ☒ ☒
DDRD	0x0A (0x2A)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PIND	0x09 (0x29)	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTD	0x0B (0x2B)	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Name	Value	Type	Location
[1]	69 'E'	unsigned char	0x0215 [SRAM]
[2]	83 'S'	unsigned char	0x0216 [SRAM]
[3]	79 'O'	unsigned char	0x0217 [SRAM]
[4]	83 'S'	unsigned char	0x0218 [SRAM]
[5]	0 ''	unsigned char	0x0219 [SRAM]
[6]	0 ''	unsigned char	0x021A [SRAM]
[7]	0 ''	unsigned char	0x021B [SRAM]
[8]	0 ''	unsigned char	0x021C [SRAM]
[9]	0 ''	unsigned char	0x021D [SRAM]
[10]	0 ''	unsigned char	0x021E [SRAM]
[11]	0 ''	unsigned char	0x021F [SRAM]
[12]	0 ''	unsigned char	0x0220 [SRAM]
[13]	0 ''	unsigned char	0x0221 [SRAM]
[14]	0 ''	unsigned char	0x0222 [SRAM]
[15]	0 ''	unsigned char	0x0223 [SRAM]
[16]	0 ''	unsigned char	0x0224 [SRAM]
[17]	0 ''	unsigned char	0x0225 [SRAM]
[18]	0 ''	unsigned char	0x0226 [SRAM]
[19]	0 ''	unsigned char	0x0227 [SRAM]
contor_20ms	0 ''	unsigned char	R13
pausa_contor	15 'I'	unsigned char	R3
temp_morse	[...]	unsigned char[6]	0x0228 [SRAM]
[0]	46 '.,'	unsigned char	0x0228 [SRAM]
[1]	46 '.,'	unsigned char	0x0229 [SRAM]
[2]	46 '.,'	unsigned char	0x022A [SRAM]
[3]	0 ''	unsigned char	0x022B [SRAM]
[4]	0 ''	unsigned char	0x022C [SRAM]
[5]	0 ''	unsigned char	0x022D [SRAM]
sos_step	0 ''	unsigned char	R14
SOS_activ	1 'I'	unsigned char	R11

Fig 3.11 mesaj special SOS

CONCLUZII

Proiectul realizat demonstrează funcționarea corectă a unui **sistem de transmitere și recepție a codului Morse**, bazat pe un automat finit de stare (**FSM**) și prelucrarea asincronă a semnalelor prin întreruperi (**ISR**). Prin utilizarea resurselor hardware ale microcontrolerului **ATmega164P**, sistemul permite compunerea mesajelor în format ASCII și transformarea lor în semnale vizuale standardizate.

În urma testelor efectuate în mediul de simulare:

- **Sistemul a reacționat corespunzător în toate stările definite** (IDLE, COMPUNERE, TRANSMISIE), asigurând o tranziție fluidă între modurile de lucru prin utilizarea butoanelor de control .
- **Transmisia a respectat temporizarea standard Morse** ($T = 100\text{ ms}$), generând puncte și linii precise pe ieșirile PORTB, cu pauze corect calibrate între simboluri, litere și cuvinte .
- **Recepția în fundal a fost validată cu succes**, sistemul fiind capabil să identifice semnalele externe pe PORTC, să le decodeze în caractere ASCII și să le stocheze într-un **buffer circular (Buffer 2)** protejat de delimitatori specifici (#) .
- **Cazul special SOS a fost gestionat prioritar**, sistemul identificând secvența critică și activând imediat semnalizarea vizuală de 500 ms timp de 10 secunde, perioadă în care funcția de transmisie a fost blocată conform cerințelor .

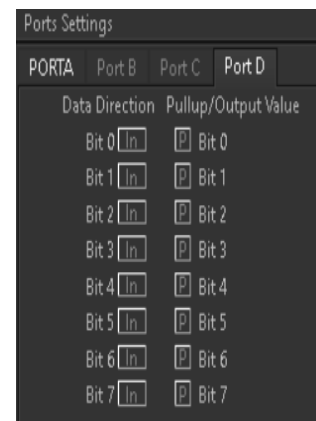
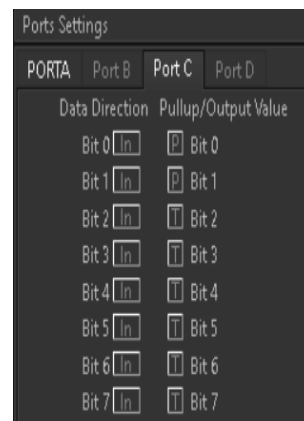
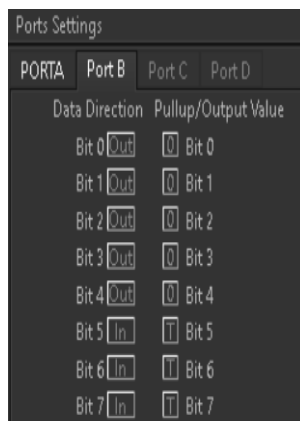
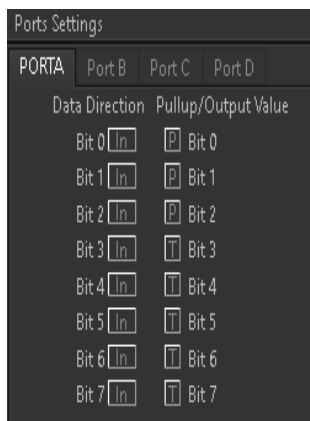
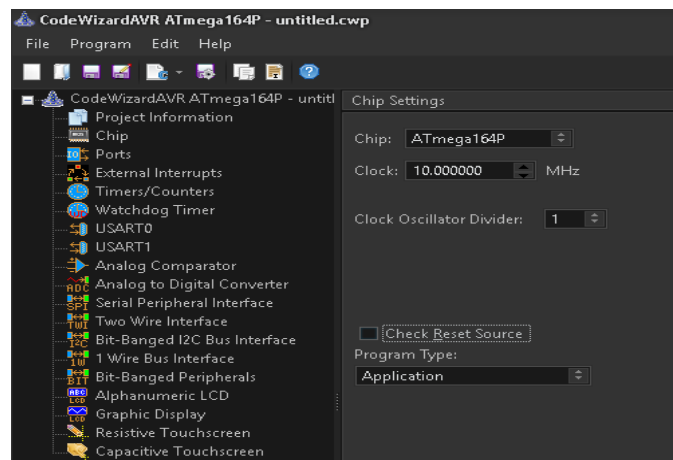
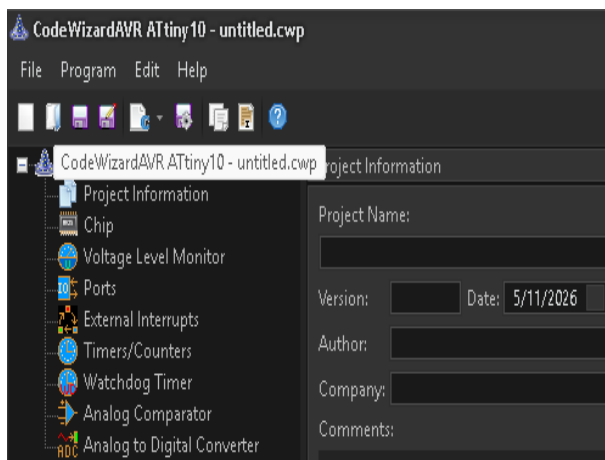
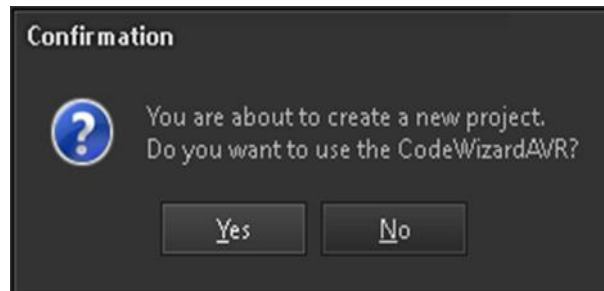
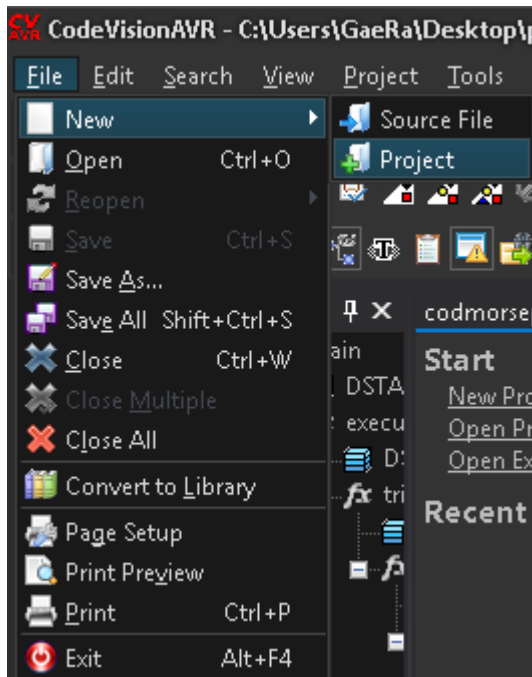
Astfel, implementarea propusă îndeplinește cerințele unui sistem de comunicație robust, oferind o funcționare **full-duplex logică** prin rularea simultană a modulelor de transmisie și recepție. Proiectul poate fi extins în viitor prin adăugarea unei interfețe de afișare LCD sau prin implementarea unei comunicații wireless între două module similare.

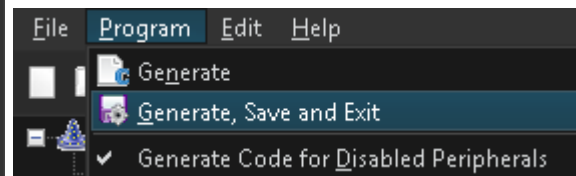
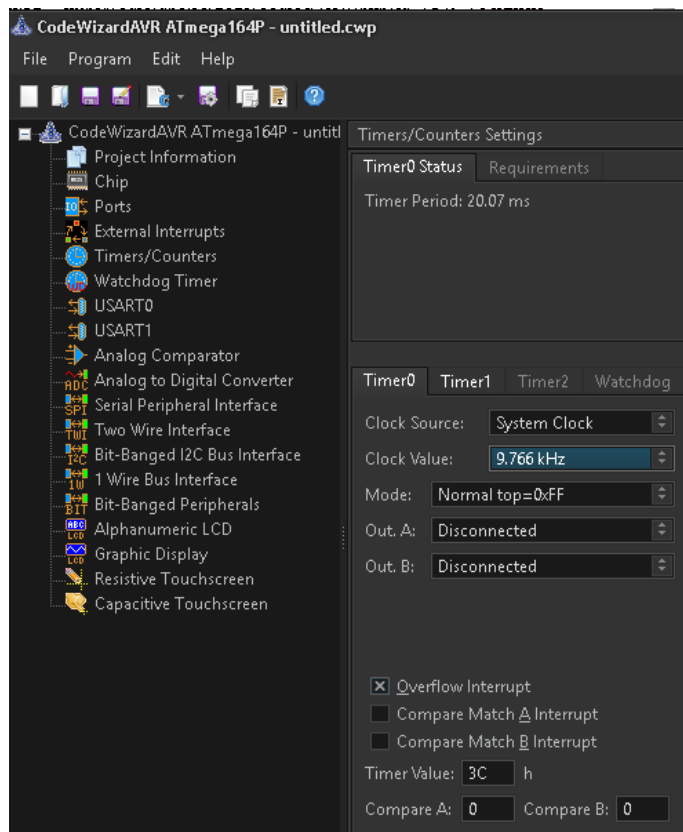
BIBLIOGRAFIE

- **Standardul Codului Morse Internațional** – <https://www.itu.int/rec/R-REC-M.1677-1-200910-I/> (Documentație privind temporizarea simbolurilor punct, linie și pauză)
- **Pagina disciplinei AMP2.MC** – <https://curs.upb.ro/2025/course/view.php?id=7349> (Cerințele generale și regulamentul de formatare pentru tema suplimentară)
- **HP InfoTech - CodeVisionAVR User Manual** – www.hpinfotech.ro/cvavr_manual.pdf (Documentația pentru utilizarea generatorului de cod și a întreruperilor **interrupt** [TIM0_OVF]).
- **Microchip Academy - AVR Timer/Counter Interrupts** – microchipdeveloper.com/avr:timer-interrupt (Ghid tehnic despre modul în care funcționează întreruperile de tip Overflow pentru eșantionare).

ANEXE

ANEXA 1





ANEXA 2

/******

This program was created by the CodeWizardAVR V4.07
Automatic Program Generator
© Copyright 1998-2026 Pavel Haiduc, HP InfoTech S.R.L.
<http://www.hpinfotech.ro>

Project : codmorse
Version :
Date : 5/10/2026
Author :
Company :
Comments:

Chip type : ATmega164
Program type : Application
AVR Core Clock frequency: 10.000000 MHz
Memory model : Small
External RAM size : 0
Data Stack size : 256

*****/

// I/O Registers definitions

#include <mega164.h>

#include <delay.h>

// Declare your global variables here

#define T_BASE 100 // Baza de timp T = 100ms

#define MAX_BUFFER 20 // Capacitate buffer

#define DELIMITATOR '#'

unsigned char Q=0; // Starea curenta a automatului (FSM)

unsigned char buffer_tx[MAX_BUFFER];

int index_tx = 0; // Index pentru Input stack

// Variabile pentru receptie (Sampling)

unsigned char buffer_rx[MAX_BUFFER];

int index_rx = 0;

unsigned char temp_morse[6]; // Buffer pentru puncte/linii (max 5 pentru cifre)

int temp_index = 0;

unsigned char pauza_contor = 0;

unsigned char mesaj_in_curs = 0;

// --- Variabile pentru cazul special SOS ---

unsigned char SOS_activ = 0; // 1 = Transmisie blocata, LED SOS clipeste

unsigned char sos_step = 0; // Monitorizarea secventei S-O-S

unsigned int timer_sos_10s = 0; // 10 secunde de activare

unsigned int clipire_500ms = 0; // Interval clipire LED

unsigned char contor_20ms=0;

// --- FUNCTII AUXILIARE TRANSMISIE ---

```

void pulse_morse(unsigned char tip) {
    if (tip == '.') {
        PORTB.0 = 1; delay_ms(T_BASE); PORTB.0 = 0; // Punct = 1T
    } else if (tip == '-') {
        PORTB.1 = 1; delay_ms(3 * T_BASE); PORTB.1 = 0; // Linie = 3T
    }
    delay_ms(T_BASE); // Pauza intre simboluri = 1T
}

void trimite_caracter(unsigned char c) {
    switch(c) {
        case 'A': pulse_morse('.'); pulse_morse('-'); break;
        case 'B': pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); break;
        case 'C': pulse_morse('-'); pulse_morse('.'); pulse_morse('-'); pulse_morse('.'); break;
        case 'D': pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); break;
        case 'E': pulse_morse('.'); break;
        case 'F': pulse_morse('.'); pulse_morse('.'); pulse_morse('-'); pulse_morse('.'); break;
        case 'G': pulse_morse('-'); pulse_morse('-'); pulse_morse('.'); break;
        case 'H': pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); break;
        case 'I': pulse_morse('.'); pulse_morse('.'); break;
        case 'J': pulse_morse('.'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); break;
        case 'K': pulse_morse('-'); pulse_morse('.'); pulse_morse('-'); break;
        case 'L': pulse_morse('.'); pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); break;
        case 'M': pulse_morse('-'); pulse_morse('-'); break;
        case 'N': pulse_morse('-'); pulse_morse('.'); break;
        case 'O': pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); break;
        case 'P': pulse_morse('.'); pulse_morse('-'); pulse_morse('-'); pulse_morse('.'); break;
        case 'Q': pulse_morse('-'); pulse_morse('-'); pulse_morse('.'); pulse_morse('-'); break;
        case 'R': pulse_morse('.'); pulse_morse('-'); pulse_morse('.'); break;
        case 'S': pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); break;
        case 'T': pulse_morse('-'); break;
        case 'U': pulse_morse('.'); pulse_morse('.'); pulse_morse('-'); break;
        case 'V': pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('-'); break;
        case 'W': pulse_morse('.'); pulse_morse('-'); pulse_morse('-'); break;
        case 'X': pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); pulse_morse('-'); break;
        case 'Y': pulse_morse('-'); pulse_morse('.'); pulse_morse('-'); pulse_morse('-'); break;
        case 'Z': pulse_morse('-'); pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); break;

        case '1': pulse_morse('.'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-');
        break;
        case '2': pulse_morse('.'); pulse_morse('.'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-');
        break;
        case '3': pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('-'); pulse_morse('-');
        break;
        case '4': pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('-');
        break;
        case '5': pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.');
        break;
        case '6': pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.');
        break;
        case '7': pulse_morse('-'); pulse_morse('-'); pulse_morse('.'); pulse_morse('.'); pulse_morse('.');
        break;
    }
}

```

```

        case '8': pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); pulse_morse('.'); pulse_morse('.');
break;
        case '9': pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); pulse_morse('.');
break;
        case '0': pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-'); pulse_morse('-');
break;

        case ' ': delay_ms(4 * T_BASE); break; // Pauza cuvant = 7T
    }
}

void executa_transmisie(void) {
    int i;
    PORTB.4 = 1; // LED TX ON

    for (i = 0; i < index_tx; i++) {
        trimite_caracter(buffer_tx[i]);
        delay_ms(2 * T_BASE); // Pauza intre litere (total 3T)
    }

    delay_ms(4 * T_BASE); // Asteptare minima 7T dupa mesaj
    PORTB.4 = 0;
    index_tx = 0;
    Q = 0;
}

unsigned char decodeaza_morse(void) {
    unsigned char rezultat = '?';

    if (temp_index == 1) {
        if (temp_morse[0] == '.') rezultat = 'E';
        else if (temp_morse[0] == '-') rezultat = 'T';
    }
    else if (temp_index == 2) {
        if (temp_morse[0] == '.' && temp_morse[1] == '-') rezultat = 'A';
        else if (temp_morse[0] == '-' && temp_morse[1] == '.') rezultat = 'N';
        else if (temp_morse[0] == '.' && temp_morse[1] == '.') rezultat = 'I';
        else if (temp_morse[0] == '-' && temp_morse[1] == '-') rezultat = 'M';
    }
    else if (temp_index == 3) {
        if (temp_morse[0] == '-' && temp_morse[1] == '.' && temp_morse[2] == '.') rezultat = 'D';
        if (temp_morse[0] == '.' && temp_morse[1] == '-' && temp_morse[2] == '.') rezultat = 'R';
        if (temp_morse[0] == '.' && temp_morse[1] == '.' && temp_morse[2] == '.') rezultat = 'S';
        if (temp_morse[0] == '-' && temp_morse[1] == '-' && temp_morse[2] == '-') rezultat = 'O';
        if (temp_morse[0] == '-' && temp_morse[1] == '.' && temp_morse[2] == '-') rezultat = 'K';
        if (temp_morse[0] == '.' && temp_morse[1] == '-' && temp_morse[2] == '-') rezultat = 'W';
        if (temp_morse[0] == '-' && temp_morse[1] == '-' && temp_morse[2] == '.') rezultat = 'G';
        if (temp_morse[0] == '.' && temp_morse[1] == '.' && temp_morse[2] == '-') rezultat = 'U';
    }
    else if (temp_index == 4) {
        if (temp_morse[0] == '-' && temp_morse[1] == '.' && temp_morse[2] == '.' && temp_morse[3] == '.')
rezultat = 'B';
    }
}

```

```

    if (temp_morse[0]=='-' && temp_morse[1]=='.' && temp_morse[2]=='-' && temp_morse[3]=='.')
rezultat = 'C';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='-' && temp_morse[3]=='.')
rezultat = 'F';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='.')
rezultat = 'H';
    if (temp_morse[0]=='.' && temp_morse[1]=='-' && temp_morse[2]=='-' && temp_morse[3]=='.')
rezultat = 'J';
    if (temp_morse[0]=='.' && temp_morse[1]=='-' && temp_morse[2]=='.' && temp_morse[3]=='.')
rezultat = 'L';
    if (temp_morse[0]=='.' && temp_morse[1]=='-' && temp_morse[2]=='-' && temp_morse[3]=='.')
rezultat = 'P';
    if (temp_morse[0]=='-' && temp_morse[1]=='-' && temp_morse[2]=='.' && temp_morse[3]=='.')
rezultat = 'Q';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='.')
rezultat = 'V';
    if (temp_morse[0]=='-' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='.')
rezultat = 'X';
    if (temp_morse[0]=='-' && temp_morse[1]=='.' && temp_morse[2]=='-' && temp_morse[3]=='.')
rezultat = 'Y';
    if (temp_morse[0]=='-' && temp_morse[1]=='-' && temp_morse[2]=='.' && temp_morse[3]=='.')
rezultat = 'Z';
}
else if (temp_index == 5) {
    if (temp_morse[0]=='.' && temp_morse[1]=='-' && temp_morse[2]=='-' && temp_morse[3]=='-'
&& temp_morse[4]=='-') rezultat = '1';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='-' && temp_morse[3]=='-' &&
temp_morse[4]=='-') rezultat = '2';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='-' &&
temp_morse[4]=='-') rezultat = '3';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='.' &&
temp_morse[4]=='-') rezultat = '4';
    if (temp_morse[0]=='.' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='.' &&
temp_morse[4]=='.') rezultat = '5';
    if (temp_morse[0]=='-' && temp_morse[1]=='.' && temp_morse[2]=='.' && temp_morse[3]=='.' &&
temp_morse[4]=='.') rezultat = '6';
    if (temp_morse[0]=='-' && temp_morse[1]=='-' && temp_morse[2]=='.' && temp_morse[3]=='.' &&
temp_morse[4]=='.') rezultat = '7';
    if (temp_morse[0]=='-' && temp_morse[1]=='-' && temp_morse[2]=='-' && temp_morse[3]=='.'
&& temp_morse[4]=='.') rezultat = '8';
    if (temp_morse[0]=='-' && temp_morse[1]=='-' && temp_morse[2]=='-' && temp_morse[3]=='-'
&& temp_morse[4]=='.') rezultat = '9';
    if (temp_morse[0]=='-' && temp_morse[1]=='-' && temp_morse[2]=='-' && temp_morse[3]=='-'
&& temp_morse[4]=='-') rezultat = '0';
}

// --- Detectie Secventiala SOS ---
if (rezultat == 'S' && (sos_step == 0 || sos_step == 2)) sos_step++;
else if (rezultat == 'O' && sos_step == 1) sos_step++;
else sos_step = 0;

if (sos_step == 3) {

```

```

    SOS_activ = 1;      // Blocheaza transmisia
    timer_sos_10s = 500; // 10 secunde (500 unitati x 20ms)
    sos_step = 0;
}

return rezultat;
}

// Timer 0 overflow interrupt service routine
interrupt [TIM0_OVF] void timer0_ovf_isr(void) {
    TCNT0 = 0x3C; // Baza de timp ~20ms

    // A. Comportament Special SOS (10s Clipire / Blocare)
    if (SOS_activ == 1) {
        if (timer_sos_10s > 0) {
            timer_sos_10s--;
            clipire_500ms++;

            // Perioada de clipire 500ms (250ms ON / 250ms OFF)
            if (clipire_500ms >= 12) {
                // Activam/Dezactivam ambele LED-uri simultan
                if (PORTB.0 == 0) {
                    PORTB.0 = 1; // LED Punct ON
                    PORTB.1 = 1; // LED Linie ON
                } else {
                    PORTB.0 = 0; // LED Punct OFF
                    PORTB.1 = 0; // LED Linie OFF
                }
                clipire_500ms = 0;
            }
        } else {
            SOS_activ = 0; // Deblocare transmisie dupa 10 secunde
            PORTB.0 = 0;   // Asiguram stingerea LED-urilor
            PORTB.1 = 0;
        }
    }

    // B. Receptie Morse (Sampling PINC)
    if (PINC.0 == 0 || PINC.1 == 0) { // Semnal activ (SAU logic)
        if (mesaj_in_curs == 0) {
            buffer_rx[index_rx] = DELIMITATOR; // Start mesaj
            index_rx = (index_rx + 1) % MAX_BUFFER; // Coadă circulară
            mesaj_in_curs = 1;
        }
        contor_20ms++;
        pauza_contor = 0;
    } else {
        if (contor_20ms > 0) {
            if (temp_index < 5) {
                if (contor_20ms >= 2 && contor_20ms <= 8) temp_morse[temp_index++] = '.'; // Punct
                else if (contor_20ms >= 12 && contor_20ms <= 20) temp_morse[temp_index++] = '-'; // Linie
            }
        }
    }
}

```

```

    contor_20ms = 0;
}

pauza_contor++;

// Pauza intre litere (3T = ~300ms)
if (pauza_contor == 15 && temp_index > 0) {
    buffer_rx[index_rx] = decodeaza_morse();
    index_rx = (index_rx + 1) % MAX_BUFFER;
    temp_index = 0;
}

// Pauza intre cuvinte (7T = ~700ms)
if (pauza_contor == 35 && mesaj_in_curs == 1) {
    buffer_rx[index_rx] = ' ';
    index_rx = (index_rx + 1) % MAX_BUFFER;
}

// Final de mesaj (Pauza lunga > 2 secunde)
if (pauza_contor > 100 && mesaj_in_curs == 1) {
    buffer_rx[index_rx] = DELIMITATOR;
    index_rx = (index_rx + 1) % MAX_BUFFER;
    mesaj_in_curs = 0;
    pauza_contor = 0;
}
}
}

void main(void)
{
    // Declare your local variables here

    // Clock Oscillator division factor: 1
    #pragma optsize-
    CLKPR=(1<<CLKPCE);
    CLKPR=(0<<CLKPCE) | (0<<CLKPS3) | (0<<CLKPS2) | (0<<CLKPS1) | (0<<CLKPS0);
    #ifdef _OPTIMIZE_SIZE_
    #pragma optsize+
    #endif

    // Input/Output Ports initialization
    // Port A initialization
    // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
    DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) |
    (0<<DDA0);
    // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=P Bit1=P Bit0=P
    PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) |
    (1<<PORTA2) | (1<<PORTA1) | (1<<PORTA0);

    // Port B initialization
    // Function: Bit7=In Bit6=In Bit5=In Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
    DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) |
    (1<<DDB0);

```

```

// State: Bit7=T Bit6=T Bit5=T Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) |
(0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) |
(0<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=P Bit0=P
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) |
(0<<PORTC2) | (1<<PORTC1) | (1<<PORTC0);

// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) |
(0<<DDD0);
// State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
PORTD=(1<<PORTD7) | (1<<PORTD6) | (1<<PORTD5) | (1<<PORTD4) | (1<<PORTD3) |
(1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);

// Timer/Counter 0 initialization
// Clock source: System Clock
// Clock value: 9.766 kHz
// Mode: Normal top=0xFF
// OC0A output: Disconnected
// OC0B output: Disconnected
// Timer Period: 20.07 ms
TCCR0A=(0<<COM0A1) | (0<<COM0A0) | (0<<COM0B1) | (0<<COM0B0) | (0<<WGM01) |
(0<<WGM00);
TCCR0B=(0<<WGM02) | (1<<CS02) | (0<<CS01) | (1<<CS00);
TCNT0=0x3C;
OCR0A=0x00;
OCR0B=0x00;

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: Timer1 Stopped
// Mode: Normal top=0xFFFF
// OC1A output: Disconnected
// OC1B output: Disconnected
// Noise Canceler: Off
// Input Capture on Falling Edge
// Timer1 Overflow Interrupt: Off
// Input Capture Interrupt: Off
// Compare A Match Interrupt: Off
// Compare B Match Interrupt: Off
TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) |
(0<<WGM10);
TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) |
(0<<CS10);
TCNT1H=0x00;
TCNT1L=0x00;

```

```

ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

// Timer/Counter 2 initialization
// Clock source: System Clock
// Clock value: Timer2 Stopped
// Mode: Normal top=0xFF
// OC2A output: Disconnected
// OC2B output: Disconnected
ASSR=(0<<EXCLK) | (0<<AS2);
TCCR2A=(0<<COM2A1) | (0<<COM2A0) | (0<<COM2B1) | (0<<COM2B0) | (0<<WGM21) |
(0<<WGM20);
TCCR2B=(0<<WGM22) | (0<<CS22) | (0<<CS21) | (0<<CS20);
TCNT2=0x00;
OCR2A=0x00;
OCR2B=0x00;

// Timer/Counter 0 Interrupt(s) initialization
TIMSK0=(0<<OCIE0B) | (0<<OCIE0A) | (1<<TOIE0);

// Timer/Counter 1 Interrupt(s) initialization
TIMSK1=(0<<ICIE1) | (0<<OCIE1B) | (0<<OCIE1A) | (0<<TOIE1);

// Timer/Counter 2 Interrupt(s) initialization
TIMSK2=(0<<OCIE2B) | (0<<OCIE2A) | (0<<TOIE2);

// External Interrupt(s) initialization
// INT0: Off
// INT1: Off
// INT2: Off
// Interrupt on any change on pins PCINT0-7: Off
// Interrupt on any change on pins PCINT8-15: Off
// Interrupt on any change on pins PCINT16-23: Off
// Interrupt on any change on pins PCINT24-31: Off
EICRA=(0<<ISC21) | (0<<ISC20) | (0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
EIMSK=(0<<INT2) | (0<<INT1) | (0<<INT0);
PCICR=(0<<PCIE3) | (0<<PCIE2) | (0<<PCIE1) | (0<<PCIE0);

// USART0 initialization
// USART0 disabled
UCSR0B=(0<<RXCIE0) | (0<<TXCIE0) | (0<<UDRIE0) | (0<<RXEN0) | (0<<TXEN0) | (0<<UCSZ02) |
(0<<RXB80) | (0<<TXB80);

// USART1 initialization
// USART1 disabled
UCSR1B=(0<<RXCIE1) | (0<<TXCIE1) | (0<<UDRIE1) | (0<<RXEN1) | (0<<TXEN1) | (0<<UCSZ12) |
(0<<RXB81) | (0<<TXB81);

```

```

// Analog Comparator initialization
// Analog Comparator: Off
// The Analog Comparator's positive input is
// connected to the AIN0 pin
// The Analog Comparator's negative input is
// connected to the AIN1 pin
ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) |
(0<<ACIS0);
ADCSRB=(0<<ACME);
// Digital input buffer on AIN0: On
// Digital input buffer on AIN1: On
DIDR1=(0<<AIN0D) | (0<<AIN1D);

// ADC initialization
// ADC disabled
ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) |
(0<<ADPS1) | (0<<ADPS0);

// SPI initialization
// SPI disabled
SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) |
(0<<SPR0);

// TWI initialization
// TWI disabled
TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

// Globally enable interrupts
#asm("sei")

while (1)
{
    switch(Q) {
        case 0: // STAREA REPAUS (IDLE)
            PORTB.2 = 1; // LED IDLE ON
            if (PINA.0 == 0) { // Apasare MOD
                delay_ms(200);
                PORTB.2 = 0;
                Q = 1; // Trecere în COMPUNERE
            }
            break;

        case 1: // --- STAREA COMPUNERE ---
            PORTB.3 = 1; // LED COMPUNERE ON

            if (PINA.1 == 0) { // Buton SAVE
                if (index_tx < MAX_BUFFER) {
                    buffer_tx[index_tx] = PIND & 0x7F; // Preluare ASCII
                    index_tx++;
                }
                delay_ms(250);
            }
    }
}

```

```

    if (PINA.2 == 0) { // Buton DELETE
        if (index_tx > 0) index_tx--;
        delay_ms(250);
    }

    if (PIND.7 == 0 && SOS_activ == 0) { // Pornire Transmisie
        delay_ms(200);
        PORTB.3 = 0;
        Q = 2;
    }

    if (PINA.0 == 0) { // Revenire manuala MOD
        delay_ms(200);
        PORTB.3 = 0;
        Q = 0;
    }
    break;

case 2: // --- STAREA TRANSMISIE ---
    executa_transmisie();
    break;
}
}
}

```